

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Case Study

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO3 – Precision	Assessment:	Assessment Tool 2 – Case Study
Weightage:	30% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	V Yogeswar	Reg. No.:	192425175
Section / Batch:		Date:	

Code of Conduct

I V Yogeswar / 192425175 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: V Yogeswar

Instructions

- Read all three case studies carefully before attempting the questions.
- Provide appropriate justifications and interpretations for every computed result.
- Use NLP concepts, algorithms, and terminology wherever applicable.
- Diagrams, flowcharts, tables, or mathematical formulations may be used to support your answers.
- Duration: 30 minutes.

Section / Topic	Focus	Case Study
N-Gram Language Models and Smoothing Techniques	Bigram and Trigram probability computation, Maximum Likelihood Estimation (MLE), Backoff models, Deleted Interpolation, Entropy calculation, uncertainty analysis, real-time next-word prediction	Case Study 1 – Smart Mobile Keyboard Prediction System
Part-of-Speech (POS) Tagging and Word Classes	Penn Treebank tagset, contextual ambiguity resolution, rule-based tagging, stochastic tagging (HMM), emission and transition probabilities, word class identification, chatbot language understanding	Case Study 2 – AI-Powered Customer Support Chatbot
Transformation-Based Tagging (Brill Tagger)	POS tag correction, transformation rules, contextual tagging, linguistic validation, tag sequence refinement, ambiguity handling	Case Study 3 – News Analytics and POS Tag Correction System
Corpus Statistics and Probabilistic Analysis	Word frequency counting, corpus-based probability estimation, entropy-based uncertainty measurement, tagging confidence evaluation	Case Study 3 – News Analytics and POS Tag Correction System
Integrated Syntactic Analysis and NLP Applications	Application of language models, POS tagging, probabilistic methods, corpus analysis, ambiguity resolution, and decision-making in real-world NLP systems	Case Studies 1, 2, and 3

CASE STUDY 1: E-Commerce Smart Search and Product Prediction System

An e-commerce company is developing an AI-powered search and autocomplete system that predicts the next word when customers enter product-related queries. The language model is trained on the corpus:

"machine learning improves business, machine learning enables automation, machine learning drives innovation."

The development team uses **Bigram and Trigram Language Models, Backoff Models, Deleted Interpolation, and Entropy Analysis** to improve search prediction and handle unseen word sequences.

The following statistics are obtained from the corpus:

- $C(\text{machine}) = 3$
- $C(\text{machine learning}) = 3$
- $C(\text{learning improves}) = 1$
- $C(\text{learning enables}) = 1$
- $C(\text{learning drives}) = 1$
- $C(\text{learning improves business}) = 1$

The interpolation weights are:

- $\lambda_1 = 0.5$ (Trigram)
- $\lambda_2 = 0.3$ (Bigram)
- $\lambda_3 = 0.2$ (Unigram)

The prediction probabilities after the word **"learning"** are:

- $P(\text{improves}) = 0.33$
- $P(\text{enables}) = 0.33$
- $P(\text{drives}) = 0.33$

Questions

1. Compute the Maximum Likelihood Estimate (MLE) of the Bigram Probability **$P(\text{learning} | \text{machine})$** . Interpret how this probability can influence the next-word prediction of an e-commerce search system.
2. A customer enters the unseen sequence **"machine learning transforms"**. Apply the Backoff Model and estimate the probability of the word **"transforms"** using lower-order N-grams. Explain why backoff is useful when customers enter previously unseen search queries.
3. Using Deleted Interpolation, calculate the probability of the sequence **"machine learning improves"** using the given interpolation weights. Explain how combining trigram, bigram, and unigram probabilities improves robustness when training data is sparse.
4. Calculate the Entropy of the prediction distribution **$P(\text{improves}) = 0.33$, $P(\text{enables}) = 0.33$, $P(\text{drives}) = 0.33$** . Interpret the entropy value and explain what it indicates about uncertainty in the search prediction system.

5. Develop a Python program using **NLTK** for N-gram generation, **NumPy** for probability computations, and the **Math** library for entropy calculation. The program should construct Unigram, Bigram, and Trigram models from the given corpus, compute the MLE Bigram probability, estimate an unseen sequence using Backoff, calculate the Deleted Interpolation probability using the given weights, compute entropy of the prediction distribution, and display all intermediate calculations and the final next-word prediction in a structured format.

1. MLE of Bigram Probability

Given:

$$C(\text{machine}, \text{learning}) = 3$$

$$C(\text{machine}) = 3$$

Formula:

$$P(\text{learning} \mid \text{machine}) = C(\text{machine}, \text{learning}) / C(\text{machine})$$

Calculation:

$$P(\text{learning} \mid \text{machine}) = 3 / 3$$

$$= 1.0$$

Interpretation:

The probability of learning occurring after machine is **1.0 (100%)**. Therefore, the search system can strongly predict **learning** as the next word after **machine**.

2. Backoff Model for "machine learning transforms"

The customer enters:

machine learning transforms

The trigram is:

(machine, learning, transforms)

This trigram does not occur in the training corpus.

Step 1 – Trigram:

$$P(\text{transforms} \mid \text{machine}, \text{learning}) = 0$$

because $C(\text{machine}, \text{learning}, \text{transforms}) = 0$.

Step 2 – Bigram:

$$P(\text{transforms} \mid \text{learning}) = 0$$

because $C(\text{learning, transforms}) = 0$.

Step 3 – Unigram:

$P(\text{transforms}) = 0$

because transforms does not occur in the corpus.

Final Backoff Probability:

$P(\text{transforms} \mid \text{machine, learning}) = 0$

Interpretation:

Backoff is useful because it allows the system to use lower-order N-grams when a higher-order N-gram is unseen. In this particular case, even the unigram is unseen, so the final probability remains **0**.

3. Deleted Interpolation

Given weights:

$\lambda_1 = 0.5$ (Trigram)

$\lambda_2 = 0.3$ (Bigram)

$\lambda_3 = 0.2$ (Unigram)

For the sequence:

machine learning improves

Trigram Probability

$C(\text{machine, learning, improves}) = 1$

$C(\text{machine, learning}) = 3$

$P(\text{improves} \mid \text{machine, learning}) = 1 / 3$

= 0.3333

Bigram Probability

$C(\text{learning, improves}) = 1$

$C(\text{learning}) = 3$

$P(\text{improves} \mid \text{learning}) = 1 / 3$

= 0.3333

Unigram Probability

Total number of words = 12

$C(\text{improves}) = 1$

$P(\text{improves}) = 1 / 12$

= 0.0833

Interpolation Formula

$P(\text{improves} \mid \text{machine, learning}) =$

$\lambda_1 \times P(\text{improves} \mid \text{machine, learning})$

- $\lambda_2 \times P(\text{improves} \mid \text{learning})$
- $\lambda_3 \times P(\text{improves})$

Calculation

$= (0.5 \times 0.3333) + (0.3 \times 0.3333) + (0.2 \times 0.0833)$

$= 0.1667 + 0.1000 + 0.0167$

= 0.2834

Final Answer: 0.2833 approximately

Interpretation:

Deleted Interpolation combines trigram, bigram, and unigram probabilities. Therefore, the model remains more robust when higher-order N-grams are sparse or unavailable.

4. Entropy of Prediction Distribution

Given:

$P(\text{improves}) = 0.33$

$P(\text{enables}) = 0.33$

$P(\text{drives}) = 0.33$

Entropy Formula:

$H = -\sum P(x) \log_2 P(x)$

Calculation:

$H = -[0.33 \log_2(0.33) + 0.33 \log_2(0.33) + 0.33 \log_2(0.33)]$

$$H = -[3 \times 0.33 \times \log_2(0.33)]$$

$H \approx 1.58$ bits

Interpretation:

The entropy is approximately **1.58 bits**, which is close to the maximum entropy for three possible outcomes. Since all three words have almost equal probabilities, the system has **high uncertainty** about which word will occur next.

5. Python Program

```

1st.py X
1st.py > ...
104 print("Trigram probability =", round(p_tri, 4))
105 print("Bigram probability =", round(p_bi, 4))
106 print("Unigram probability =", round(p_uni, 4))
107
108 print("Final probability =",
109       round(p_interpolation, 4))
110
111 # -----
112 # 4. Entropy
113 # -----
114
115 print("\n4. ENTROPY")
116
117 probabilities = np.array([0.33, 0.33, 0.33])
118
119 entropy = -sum(
120     p * math.log2(p)

```

```

Problems  Output  Debug Console  Terminal  Ports

PS M:\NLP\CO3_AT2> & C:\Users\Y0geswar\AppData\Local\Programs\Python\Python313\python.exe m:/NLP/CO3_AT2/1st.py
C(machine) = 3
P(learning | machine) = 1.0

2. BACKOFF MODEL
Trigram count = 0
Bigram count = 0
Unigram count = 0
Backoff level = Unseen
P(transforms | machine learning) = 0

3. DELETED INTERPOLATION
Trigram probability = 0.3333
Bigram probability = 0.3333
Unigram probability = 0.0833
Final probability = 0.2833

4. ENTROPY
P(improves) = 0.33
P(enables) = 0.33
P(drives) = 0.33
Entropy = 1.5835 bits

5. NEXT-WORD PREDICTION
improves    0.33
enables     0.33
drives      0.33

Final predicted word: improves
PS M:\NLP\CO3_AT2>

```

CASE STUDY 2: AI-Powered Hospital Appointment Chatbot

A healthcare organization develops an AI-powered chatbot that assists patients in booking appointments. The chatbot performs **Part-of-Speech tagging** before identifying user intentions and generating responses.

Consider the following user inputs:

1. **"Book an appointment with the doctor."**
2. **"The book contains medical information."**

The chatbot uses the **Penn Treebank POS Tagset** and an **HMM-based probabilistic POS tagger** to determine the grammatical role of ambiguous words.

For the word **"book"**, the system is given:

- $P(\text{book} \mid \text{VB}) = 0.7$
- $P(\text{book} \mid \text{NN}) = 0.3$
- $P(\text{Start} \rightarrow \text{VB}) = 0.6$
- $P(\text{Start} \rightarrow \text{NN}) = 0.4$

The system must determine whether **"book"** functions as a Verb (VB) or Noun (NN) depending on its context.

Questions

1. Assign appropriate **Penn Treebank POS tags** to every word in both sentences. Justify the POS tag assigned to **"book"** using the grammatical context of each sentence.
2. Using the given HMM probabilities, compute the likelihood of **"book"** being tagged as a Verb (VB) and as a Noun (NN). Determine the most probable tag and explain why the probabilistic model favors that tag at the beginning of a sentence.
3. Compare **Rule-Based POS Tagging** and **Stochastic HMM Tagging** for resolving the ambiguity of the word **"book"**. Discuss the advantages and limitations of both approaches in a healthcare chatbot.
4. Evaluate how standardized POS tagsets such as the **Penn Treebank Tagset** can support downstream NLP tasks such as **intent detection, entity identification, and response generation** in healthcare chatbots.
5. Develop a Python program using **NLTK** to tokenize the given sentences and assign Penn Treebank POS tags. Implement a simple **HMM-based POS tagging calculation** using the given emission and transition probabilities. The program should calculate the probability of **"book"** being tagged as VB and NN, determine the most likely tag, display the tokenized sentences, POS-tagged output, intermediate probability calculations, and final tagging results in a structured format.

1 Sentence:-

Word	POS Tag
Book	VB
an	DT

Word	POS Tag
appointment	NN
with	IN
the	DT
doctor	NN
.	.

2 Sentence:-

Word	POS Tag
The	DT
book	NN
contains	VBZ
medical	JJ
information	NN
.	.

2. HMM Probability Calculation

Probability of "book" as Verb

Given:

$$P(\text{book} \mid \text{VB}) = 0.7$$

$$P(\text{Start} \rightarrow \text{VB}) = 0.6$$

Formula:

$$P(\text{VB}, \text{book}) = P(\text{book} \mid \text{VB}) \times P(\text{Start} \rightarrow \text{VB})$$

Calculation:

$$= 0.7 \times 0.6$$

$$= 0.42$$

Probability of "book" as Noun

Given:

$$P(\text{book} \mid \text{NN}) = 0.3$$

$$P(\text{Start} \rightarrow \text{NN}) = 0.4$$

Calculation:

$$= 0.3 \times 0.4$$

$$= 0.12$$

Comparison

VB probability = 0.42

NN probability = 0.12

Since:

$$0.42 > 0.12$$

Most probable tag = VB

Interpretation:

At the beginning of a sentence, the given HMM probabilities favor VB because the combined emission and transition probability for VB is higher than that for NN.

3. Rule-Based POS Tagging vs Stochastic HMM Tagging

Rule-Based POS Tagging

Advantages:

- Simple to understand.
- Easy to implement.
- Rules are transparent.
- Works well for known grammatical patterns.

Limitations:

- Requires manually created rules.
- Difficult to handle complex language.
- May fail when context changes.

Stochastic HMM Tagging

Advantages:

- Uses probabilities.
- Can use contextual information.
- Can resolve ambiguity statistically.
- Suitable for large amounts of text.

Limitations:

- Requires training data.

- Performance depends on probability estimates.
- May struggle with unseen words and contexts.

Conclusion:

For a healthcare chatbot, combining grammatical rules with probabilistic context can provide more reliable POS tagging.

4. Importance of Penn Treebank POS Tags

Standardized POS tags provide consistent grammatical information to NLP systems.

Intent Detection

POS tags can help identify actions such as:

- Book → VB
- appointment → NN

This helps identify an appointment-booking intent.

Entity Identification

POS information can help distinguish nouns that may represent entities such as:

- doctor
- hospital
- appointment
- medicine

Response Generation

POS tags help the chatbot understand the grammatical structure of the user's request and generate an appropriate response.

Conclusion:

A standardized tagset improves consistency between POS tagging and downstream NLP tasks.

5. Python Program

```
2nd.py X
2nd.py > ...
20 | print(tokens, tags)
21 | print("POS Tags:", tags)
22 |
23 | # -----
24 | # HMM Calculation
25 | # -----
26 |
27 | print("\nHMM PROBABILITY CALCULATION")
28 | print("-----")
29 |
30 | p_book_vb = 0.7
31 | p_book_nn = 0.3
32 |
33 | p_start_vb = 0.6
34 | p_start_nn = 0.4
35 |
36 | prob_vb = p_book_vb * p_start_vb

Problems Output Debug Console Terminal Ports
PS M:\NLP\CO3_AT2> python -m nltk.downloader punkt_tab
PS M:\NLP\CO3_AT2> python 2nd.py
TOKENIZATION AND POS TAGGING
-----

Sentence: Book an appointment with the doctor.
Tokens: ['Book', 'an', 'appointment', 'with', 'the', 'doctor', '.']
POS Tags: [('Book', 'NNP'), ('an', 'DT'), ('appointment', 'NN'), ('with', 'IN'), ('the', 'DT'), ('doctor', 'NN'), ('.', '.')]

Sentence: The book contains medical information.
Tokens: ['The', 'book', 'contains', 'medical', 'information', '.']
POS Tags: [('The', 'DT'), ('book', 'NN'), ('contains', 'VBZ'), ('medical', 'JJ'), ('information', 'NN'), ('.', '.')]

HMM PROBABILITY CALCULATION
-----
P(book | VB) = 0.7
P(Start -> VB) = 0.6
VB probability = 0.42

P(book | NN) = 0.3
P(Start -> NN) = 0.4
NN probability = 0.12

Most probable tag at sentence start: VB

FINAL CONTEXTUAL TAGGING
-----
Book an appointment with the doctor. -> book/VB
The book contains medical information. -> book/NN
PS M:\NLP\CO3_AT2>
```

CASE STUDY 3: Financial News POS Tag Correction and Uncertainty Analysis

A financial news analytics company processes thousands of financial news articles every day. The system initially assigns POS tags using a statistical POS tagger and subsequently applies a **Transformation-Based Tagger (Brill Tagger)** to correct common tagging errors.

The sentence processed by the system is:

"Market growth drives investment."

The initial POS tags produced by the system are:

- market/NN
- growth/NN
- drives/NNS
- investment/NN

A learned transformation rule states:

Change NNS to VBZ if the preceding word is tagged as NN.

The system also performs corpus-based frequency analysis and entropy-based uncertainty evaluation.

The corpus contains the following word frequencies:

1. market – 500 occurrences
2. growth – 350 occurrences
3. drives – 180 occurrences
4. investment – 420 occurrences

Questions

1. Apply the given transformation rule to the initial POS-tag sequence. Display the corrected POS-tag sequence and explain why the transformation is linguistically appropriate.
2. Analyze the initial tagging error for "**drives**". Explain why the word should receive the **VBZ** tag in this sentence and discuss how the surrounding grammatical context helps identify the correct tag.
3. Using the given corpus statistics, calculate the **relative frequency/probability** of each word. Explain how corpus frequency information can support probabilistic POS tagging and language processing.
4. Calculate the **entropy of the word-frequency distribution** before and after applying the transformation rule. Explain what entropy indicates about uncertainty in the tagging or prediction process.
5. Develop a Python program using **NLTK** for initial POS tagging and implement a simple **Transformation-Based Tagger** that applies the rule "**Change NNS to VBZ if the preceding word is tagged as NN.**" The program should tokenize the sentence, generate initial POS tags, apply the transformation rule, calculate word-frequency probabilities from the given corpus statistics, calculate entropy using the Python **math** library, and display the initial tags, corrected tags, frequency distribution, entropy values, and final POS-tagging results in a structured format.

1. Applying the Transformation Rule

Initial POS tags:

market/NN

growth/NN

drives/NNS

investment/NN

Transformation Rule:

Change NNS to VBZ if the preceding word is tagged as NN.

The word before **drives** is:

growth/NN

The current tag of **drives** is:

drives/NNS

Therefore, the rule applies.

Corrected POS sequence:

market/NN

growth/NN

drives/VBZ

investment/NN

Final sentence:

market/NN growth/NN drives/VBZ investment/NN

Interpretation:

The transformation is linguistically appropriate because **market growth** forms a singular noun phrase and **drives** functions as the main singular present-tense verb.

2. Error Analysis of **drives**

The initial tag was:

drives/NNS

NNS means **plural noun**.

However, in:

Market growth drives investment.

market growth is the subject.

drives is the main verb describing the action performed by the subject.

Therefore, the correct tag is:

drives/VBZ

VBZ represents a **third-person singular present-tense verb**.

The surrounding context helps identify that `drives` is functioning as a verb rather than a noun.

3. Relative Frequency / Probability

Given frequencies:

$C(\text{market}) = 500$

$C(\text{growth}) = 350$

$C(\text{drives}) = 180$

$C(\text{investment}) = 420$

Total frequency:

$\text{Total} = 500 + 350 + 180 + 420$

= 1450

Market

$P(\text{market}) = 500 / 1450$

= 0.3448

Growth

$P(\text{growth}) = 350 / 1450$

= 0.2414

Drives

$P(\text{drives}) = 180 / 1450$

= 0.1241

Investment

$$P(\text{investment}) = 420 / 1450$$

$$= \mathbf{0.2897}$$

Final Distribution:

$$\text{market} = 0.3448$$

$$\text{growth} = 0.2414$$

$$\text{drives} = 0.1241$$

$$\text{investment} = 0.2897$$

Interpretation:

Corpus frequency provides statistical information about how commonly words occur. A probabilistic POS tagger can use such frequency information along with contextual and transition probabilities to make better tagging decisions.

4. Entropy of Word-Frequency Distribution

Entropy Formula:

$$H = -\sum P(x) \times \log_2(P(x))$$

Using the probabilities:

$$H = -[(0.3448 \times \log_2(0.3448)) + (0.2414 \times \log_2(0.2414)) + (0.1241 \times \log_2(0.1241)) + (0.2897 \times \log_2(0.2897))]$$

$$H = \mathbf{1.9161 \text{ bits}}$$

Before Transformation

$$\text{Entropy} = \mathbf{1.9161 \text{ bits}}$$

After Transformation

The transformation only changes the POS tag of `drives` from NNS to VBZ. It does not change the word frequencies.

Therefore:

Entropy = **1.9161 bits**

Final Answer:

Entropy before transformation = **1.9161 bits**

Entropy after transformation = **1.9161 bits**

Interpretation:

Entropy measures the uncertainty in the word-frequency distribution. The entropy does not change because the transformation changes only the POS tag and not the word-frequency values. The transformation improves the **POS-tagging accuracy**, but it does not change the given frequency distribution.

5. Python Program

```
3rd.py X
3rd.py > ...
77 word, count in frequencies.items():
78
79     probability = count / total
80     probabilities[word] = probability
81
82     print(
83         f"{word:<12} "
84         f"Frequency = {count:<5} "
85         f"Probability = {probability:.4f}"
86     )
87
88 # -----
89 # Entropy
90 # -----
91
92 entropy_before = -sum(
93     p * math.log2(p)
```

Problems Output Debug Console **Terminal** Ports

```
PS M:\NLP\C03_AT2> & C:\Users\Y0geswar\AppData\Local\Programs\Python\Python313\python.exe m:/NLP/C03_AT2/3rd.py
drives/NNS
investment/NN

CORRECTED POS TAGS
-----
market/NN
growth/NN
drives/VBZ
investment/NN

WORD FREQUENCY DISTRIBUTION
-----
market      Frequency = 500   Probability = 0.3448
growth      Frequency = 350   Probability = 0.2414
drives      Frequency = 180   Probability = 0.1241
investment   Frequency = 420   Probability = 0.2897

ENTROPY ANALYSIS
-----
Entropy before transformation: 1.9161 bits
Entropy after transformation : 1.9161 bits

FINAL RESULT
-----
market/NN
growth/NN
drives/VBZ
investment/NN
PS M:\NLP\C03_AT2> |
```

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Q. No. / Criteria Level	Understanding of Case	Application of Concepts	Analysis	Solution Design	Clarity	Sub. Total	Total %
1	3	3	3	3	3	73	73
2	3	3	3	3	3	73	
3	3	3	3	3	3	73	

Faculty In-charge	Course Coordinator	Program Director
KUMARAGURUBARAN. T		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____